

S3 Lab Work

An evidence-traceable review system for Indian cars and two-wheelers. Every number on the site opens the sentences it was computed from.

SUBMISSION

S3 Lab work

DUE

Friday 2 October 2026

TEAM

Aditya Nariyapara, Devika Jonjale, Saachi Shinde

MEASURED

18 September 2026, on `main`

LIVE APPLICATION

revix-reviews.vercel.app

REPOSITORY

github.com/saaachis/revix

What this report covers

Five sections, in the order of the assessment breakdown.

1	Codebase	Shape, scale, and the one rule	3–4
2	Frameworks, with justification	Choices and what we rejected	5–6
3	Code quality, measured	Radon scores, and what changed	7–9
4	Non-functional requirements	All nine, audited	10–12
5	Run it live	Reproduce every figure	13

ASSESSMENT BREAKDOWN

CODEBASE	10
17,600 lines	
Three Python packages and a Next.js app, on real data	
FRAMEWORKS	10
9 decision records	
Every choice written down, including what we rejected	
CODE QUALITY	10
Grade A, all 43 files	
Measured with Radon, enforced by Xenon in CI	
NFRS ACHIEVED	10
9 of 9 audited	
Six met by one decision, five gaps found and closed	

HOW TO READ THE NUMBERS

Every figure in this document was produced by running the tool named beside it. Page 13 gives the commands, so any number here can be checked in under a minute rather than taken on trust.

Codebase

A uv workspace: three Python packages and one Next.js application, running unattended on reviews collected from three sources.

SHAPE

revix/

└─ packages/revix_core/

└─ pipeline/

└─ apps/api/

└─ apps/web/

└─ tests/

└─ db/migrations/

└─ docs/

└─ .github/workflows/

models, settings, session

ingest → fuse → publish

serving layer + operations

Next.js 16 frontend

16 modules, 308 tests

Alembic, reversible

ADRs, audits, this report

CI, nightly, probe, keep-warm

1,312

6,716

1,516

5,069

2,990

58 Python files · 24 TypeScript files · ~17,600 lines

RUNNING ON REAL DATA, NOT FIXTURES

Evidence units collected	20,263
Variants in the catalogue	143
Variants with a published verdict	134
Held back below the evidence floor	9
Models · manufacturers	42 · 16
Last full pipeline run	53m 28s

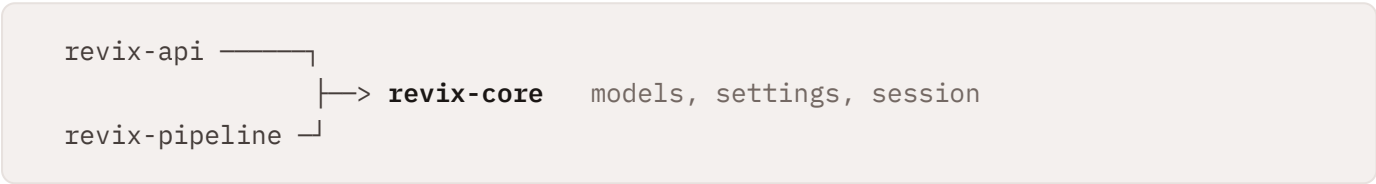
Owner reviews from CarWale and BikeWale, CarDekho and BikeDekho, plus comments on YouTube review videos. Three sources because three permit us: Team-BHP returns 403 to our agent and two other sites disallow us outright in `robots.txt`, and we chose to respect that rather than spoof a browser.

Structure that a build can enforce

THE ONE RULE THAT HOLDS IT TOGETHER

Dependencies run in **one direction only**. The API and the pipeline both depend on `revix_core`; core depends on neither, so the read path and the write path cannot tangle.

Enforced by import linting in CI, not by us remembering. In a single package this would be a naming convention, and naming conventions lose. This is also why it is three packages rather than one: the API physically cannot import the pipeline.



THE DATABASE MIRRORS THE SAME IDEA

Four Postgres schemas separated by lifecycle rather than by subject, so it is always obvious which layer a table belongs to and what is allowed to write to it.

SCHEMA	HOLDS	WRITTEN BY
raw	Fetches payloads, content-hashed	Ingest
core	Catalogue, sources, evidence units	Ingest and resolution
analysis	Aspect scores, evaluation runs	Enrichment and fusion
serving	Finished verdicts, claims, citations	Fusion; read by the API

SCHEMA CHANGES ARE CHECKED THREE WAYS

- The migrations **apply** from empty.
- They **match the models**, so editing a model without a migration is a red build.
- They **reverse cleanly**, which is the one most projects never test.

Frameworks, with justification

The artefact for this mark is the decision-record folder: nine ADRs, each stating the decision, the alternatives considered, and the consequence we accepted.

CHOICE	WHY THIS ONE	WHAT WE DID NOT DO
FastAPI	Contract-first. The TypeScript client is generated from the OpenAPI schema, so the two <i>cannot</i> drift	Flask, and hand-syncing the client
Pydantic v2	Validation at the boundary, so a malformed request never reaches a query	Manual checks scattered through handlers
SQLAlchemy 2.0	Typed ORM with bound parameters everywhere, so injection has no surface	Raw SQL: quicker to write, impossible to refactor safely
PostgreSQL + pgvector + pg_trgm	One datastore covers relational, vector <i>and</i> fuzzy text. Resolution needs trigrams; deduplication needs vectors	A separate vector database, a second service to keep consistent
Next.js 16 App Router	Server rendering, so a verdict is a real URL that can be shared and indexed	A client-only single-page app
Tailwind v4	CSS-first theme, so the palette lives in one place. It was reworked once for contrast, and one place is why that was cheap	Ad-hoc stylesheets per component
Typer	The pipeline is a CLI, so every stage is runnable and debuggable on its own	A monolithic script

THE THREAD RUNNING THROUGH ALL OF THEM

Each of these was chosen so that a mistake becomes a failed build rather than a defect a reader finds. The generated client, the typed ORM, the migration checks and the contract-first schema are all the same move: **make the wrong thing impossible to merge** rather than rely on noticing it.

The two choices worth arguing about

ADR 0002 · ORCHESTRATION

Scheduled GitHub Actions instead of Airflow or Prefect. An orchestrator is a server to run, secure and pay for. Our pipeline is one nightly batch: no fan-out, no backfill, no dynamic DAG. Actions already holds our secrets, already runs on a schedule, and already gives us logs and alerting.

Choosing Airflow here would have been choosing a tool because it looks professional rather than because the problem needs it. If we ever needed backfill or fan-out, that ADR is where the decision gets revisited.

ADR 0004 · MACHINE LEARNING, AND THE HONEST RESULT

We built a distant-supervision aspect classifier: lexicon labels train a TF-IDF into a one-vs-rest logistic regression.

It lost to the lexicon it learned from by 0.29 macro F1, so it ships disabled and the number is published on our public metrics page rather than quietly dropped.

The alternative was shipping a model because we had built one, and the whole argument of this project is that a number should be earned.

THE NINE DECISION RECORDS

0001	Record architecture decisions
0002	Scheduled CI instead of Prefect
0003	A uv workspace with three packages
0004	A lexicon baseline before a classifier
0005	The frontend stack, and why the switch is instant
0006	Official APIs only, and what we checked before deciding
0007	How the fusion experiment avoids fooling us
0009	Two sources, not three
0010	Basic auth for the operations surface

Code quality, measured

Put through a review toolkit rather than described in prose. **Radon** produces the score, **Xenon** enforces it, **Ruff** and **mypy** cover lint and types, **PyTest** runs the suite.

A

• all 43 files

MAINTAINABILITY INDEX

Radon. Every file grade A, no exceptions

3.90

AVERAGE CYCLOMATIC COMPLEXITY

Grade A, across 344 blocks

0

BLOCKS GRADED E OR F

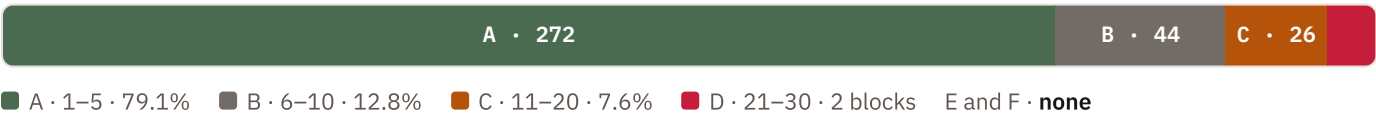
Was one before this review. See page 8

308 • 64%

TESTS • COVERAGE

Ruff, format and mypy strict all clean

COMPLEXITY DISTRIBUTION, 344 BLOCKS



WHAT EACH TOOL MEASURES

TOOL	MEASURES	RESULT
Radon	Maintainability index, cyclomatic complexity, raw counts, Halstead	A throughout
Xenon	The same measurement, wired to an exit code	gate passes
Ruff	Lint and formatting, one tool instead of three	clean
mypy	Static types, strict, all three packages	clean
PyTest	308 tests, coverage measured in CI	64%

The measurement changed the code

THE ONLY E GRADE IN THE CODEBASE

One function scored **E (33)**, forty percent worse than the next worst block. Reading it, the cause was plain: `_evaluate` drew samples from the corpus *and* turned those observations into summary statistics. Neither half is complicated. Doing both in one place was.

Split along that seam into five functions — `_draw`, `_score_one_draw`, `_rank_correlation`, `_calibration` and `_summarise` — with a small value object carrying observations between the two halves. The thirteen tests covering it passed unchanged throughout, which is the only reason a refactor there was safe to make.

BEFORE AND AFTER

	BEFORE	AFTER
Worst block	E (33)	D (24)
Blocks at E or worse	1	0
Average complexity	3.944	3.895

METHOD, NOT LUCK

Measure, find the worst thing, understand *why* it is the worst thing, fix the one that is safe to fix, and document the ones that are not.

THE TWO BLOCKS STILL AT D, DELIBERATELY

BLOCK	WHY THE COMPLEXITY IS THE DOMAIN'S, NOT A STYLE ACCIDENT
<code>fuse_variant</code> D (24)	The scoring engine: weighting, bootstrap interval, evidence floor, suppression, citations. Splitting it into five functions that only ever run in sequence would move the complexity into the call graph rather than remove it. It is the most heavily tested function in the project.
<code>run_connector</code> D (21)	The ingest loop, holding three failure modes open at once so that one source failing never stops the others. That branching is the resilience requirement.

Both sit on the critical write path, and a refactor that changes a verdict is worse than a D grade. A threshold should be set where the code is and then defended, not chased.

Enforcing the score, and the tool we left out

REPORTING IS NOT ENFORCING

Radon will tell you a function scores E and then let you merge it. That is a report, not a control. Xenon runs on every pull request and **fails the build** on any regression:

```
xenon --max-absolute D --max-modules B --max-average A
```

THRESHOLD	SET TO	WHY
Any single block	D	Where the code is; nothing may get worse than the two on page 8
Any single module	B	A module may contain a hard function, not become a hard module
The whole codebase	A	The average must stay in the simplest band

The grades here **cannot silently decay**: the next pull request that makes them worse does not merge. That is the difference between a measurement and a property.

MUTATION TESTING, AND WHY NOT

Mutmut changes your code on purpose and asks whether any test notices. **It has no native Windows support**, and all three of us develop on Windows. A quality tool only one environment can run is one that stops being run.

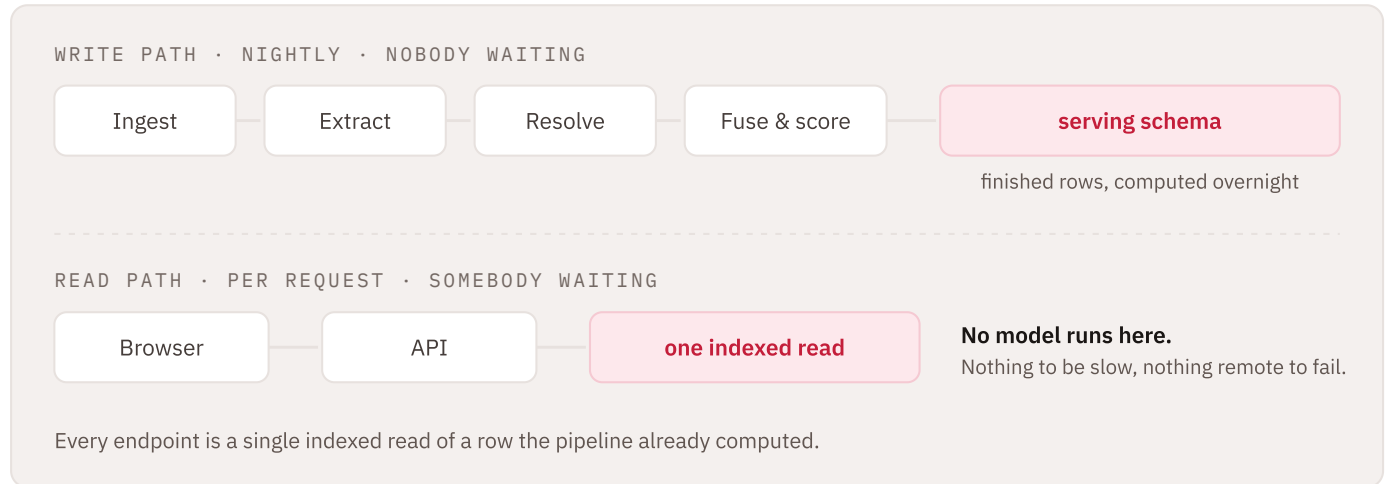
WHAT WE ANSWER INSTEAD

DEFECT	WHY IT MATTERED
Macro-F1 averaged over zero-support aspects	A perfect system scored 0.22. Our evaluation flattered us
A connector recorded our own search term as the source's answer	We would have been citing ourselves
An unordered <code>LIMIT</code> in fusion	Ingest and fusion disagreed about which variant they meant
Security headers added inside the rate limiter	A 429, the response most likely to reach a hostile client, went out bare
A migration emitting <code>NOT NULL</code> with no default	Would have failed against 129 production rows

Non-functional requirements

All nine categories audited against the actual code rather than against what the proposal claimed. **Six were already satisfied by a single architectural decision**, and that decision is worth explaining before the list.

THE WRITE PATH AND THE READ PATH ARE STRICTLY SEPARATED



22–35 ms

WARM RESPONSE, LOCAL

Against a 300 ms p95 target

96–247 ms

FROM INDIA, DEPLOYED

Inside target including the network

That one decision delivers four requirements at once. **Performance**, because nothing on the read path grows with the corpus, since the corpus is not on the read path. **Scalability**, because going from twenty thousand review units to a hundred thousand changes how long the night takes and changes nothing about a request. **Reliability**, because a page cannot fail when a third-party model is down. And **availability**, because a demo depends on nothing remote.

All nine, and where each is met

1 • Performance

met

22–35 ms warm, 96–247 ms from India against a 300 ms target. Gzip 9.3×, cache directives, and `X-Response-Time-ms` on every response so the claim is checkable from outside

2 • Scalability

met

Pagination capped at 200, bounded connection pool, the expensive work batched overnight where nobody is waiting

3 • Portability

met

Docker image 364 MB running as a non-root user, uv lockfile, configuration only through environment variables. Developed on Windows, built and tested on Linux

4 • Usability

audited

WCAG 2.1 AA clean on **all ten pages**, axe-core enforced in CI. The palette was reworked once because contrast failed, and the audit is what caught it

5 • Compatibility

met

The TypeScript client is generated from the live schema, so the frontend cannot drift from the API. Responsive layout, standards only

6 • Security

met

CSP and HSTS, CORS allow-list, rate limiting, no raw SQL, no secrets in a public repo, read-only public API and an authenticated operations surface

7 • Reliability

met

Sources degrade independently behind a circuit breaker, every database wait is bounded, and no traceback ever reaches a caller

8 • Maintainability

met

Strict types, 308 tests, nine decision records, a generated client, and configuration in one place

9 • Availability

stated honestly

A 503 health contract and a keep-warm schedule. One free-tier instance is *resilient to its dependencies failing*, not *highly available*

MEASURED, NOT ASSERTED

Accessibility 0 violations across ten pages · catalogue payload 55.3 kB down to 5.9 kB gzipped · API image 364 MB, non-root · thirteen tests in `test_nfr.py` assert the headers, the limiter, the error shape and the compression, because these are exactly the things that work the day they are added and quietly stop working six commits later.

The finding worth leading with

A REQUIREMENT WE HAD WRITTEN, TESTED, AND STILL NOT MET

Our `/health` endpoint is deliberately built to report an unreachable database as a **503** rather than a 500 with a stack trace. It was written that way on purpose. There is a test for it.

It could not actually do it. There was no connect timeout, so the driver sat on the operating system's TCP timeout while the hosting platform's health probe gave up first. A correct 503 became no answer at all — and a hang is worse than an error, because nothing downstream can react to it.

What led us there was our own test run hanging for four hundred seconds against a machine with no database. Every wait is now bounded. The lesson is the difference between *writing* a non-functional requirement and *verifying* one: we had the code, the intent and the test, and the requirement was still not met.

TWO LIMITS WE STATE RATHER THAN HIDE

LIMIT	WHY IT IS THE RIGHT CALL AT THIS SIZE
The rate limiter is per instance and in memory	It bounds what one instance absorbs rather than enforcing a global quota, which fits the actual risk: one scraper exhausting a connection pool, not a distributed attack. A global quota needs Redis, and adding a network dependency to the read path to protect the read path is a bad trade here.
A single free-tier instance is resilient, not highly available	There is no redundancy and no failover; buying that means two instances and a load balancer. Our mitigations target the failure that actually threatens a demo, a cold start or a sleeping database, rather than the one that does not.

Both are the first things that would need replacing if Revix grew. Stating a limit before being asked reads as engineering judgement; being caught at it does not.

Run it live

Every figure in this document is reproducible in under a minute. A number produced in front of you is worth more than a number in a document.

THE SCORE

```
# maintainability index, per file
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s

# cyclomatic complexity: the average, plus anything above B
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC

# raw counts
uv run radon raw packages/revix_core/src pipeline/src apps/api/src -s
```

THE GATE CI RUNS

```
uv run xenon --max-absolute D --max-modules B --max-average A \
  packages/revix_core/src pipeline/src apps/api/src
```

THE REST OF THE QUALITY GATE

```
uv run ruff check . && uv run ruff format --check .
uv run mypy packages/revix_core/src pipeline/src apps/api/src
uv run pytest --cov --cov-report=term
cd apps/web && npm run all
```

LIVE SURFACES

[Live site](#)
[API health](#)
[Source health](#)
[OpenAPI](#)
[Repository](#)

SUPPORTING DOCUMENTS

[docs/s3-lab-work/03-code-review-report.md](#) — the full toolkit report

[docs/non-functional-requirements.md](#) — all nine categories in detail

[docs/adr/](#) — the nine architecture decision records

Revix · S3 Lab Work · measured 18 September 2026 on [main](#) . Figures grow with every pipeline run; the commands above reproduce all of them.